Propuesta de Solución: Software Web de Gestión de Inventario y Ventas para una Botica

Curso: Fullstack Developer Software – Tecnologías de la Información (SENATI) **Caso:** Botica "Nova Salud"
Frontend: React · **Backend:** Node.js + Express · **BD:** PostgreSQL

> **Nota sobre el enunciado:** el caso práctico menciona la botica "**Nova Salud**", pero las preguntas guía hablan de "**FarmaPlus**". Parece un error del documento. En este trabajo se usa **Nova Salud** y conviene aclararlo en la introducción (o consultarlo con el docente).

---

1. Introducción

La botica Nova Salud creció en clientes, pero controla stock y atención de forma manual. Esto genera errores, desabastecimientos y colas largas. La propuesta es un **sistema web centralizado** que gestione inventario, ventas y atención al cliente, con actualización de stock en tiempo real y alertas automáticas de reposición.

1.1 Objetivo general

Implementar una aplicación web fullstack que automatice el control de inventario y el registro de ventas, reduciendo errores y tiempos de atención.

1.2 Objetivos específicos

- Centralizar inventario, ventas y clientes en una sola plataforma.
- Actualizar el stock automáticamente en cada venta o ingreso de mercadería.
- Generar alertas cuando un producto llegue al stock mínimo o esté por vencer.
- Diseñar una interfaz de punto de venta (POS) rápida e intuitiva.
- Garantizar la seguridad de los datos (autenticación, roles, cifrado).

---

2. Respuestas a las preguntas guía

Pregunta 1. ¿Cuáles son los principales desafíos operativos sin un sistema automatizado?

- **Errores humanos** al anotar entradas y salidas (stock físico ≠ stock registrado).
- **Desabastecimiento frecuente**: nadie detecta a tiempo que un producto se acaba.
- **Largos tiempos de espera**: buscar precio y existencias a mano retrasa la atención.
- **Productos vencidos** que no se controlan por fecha de vencimiento (crítico en farmacia).
- **Falta de trazabilidad**: no se sabe quién vendió qué ni cuándo.
- **Sin información para decidir**: no hay reportes de productos más vendidos ni de rotación.

Pregunta 2. ¿Qué características debe incluir el software?

Módulo	Características
Autenticación	Login, roles (Administrador, Vendedor, Almacén), sesiones seguras
Inventario	CRUD de productos, categorías, lotes, fecha de vencimiento, stock mínimo, ingreso

	de mercadería
Ventas (POS)	Búsqueda rápida, carrito, descuentos, métodos de pago, comprobante, descuento automático de stock
Alertas	Stock bajo, productos por vencer, productos agotados, panel de notificaciones
Clientes	Registro, historial de compras, búsqueda por DNI
Proveedores	Registro y órdenes de compra / reposición
Reportes	Ventas por día/mes, productos más vendidos, movimientos de inventario

Pregunta 3. ¿Cómo impacta en la reducción del desabastecimiento y la mejora del servicio?

- El **stock se descuenta en tiempo real** con cada venta, así que la información siempre está actualizada.
- Las **alertas de stock mínimo** permiten reponer *antes* de quedarse sin producto.
- El **POS con búsqueda instantánea** reduce el tiempo por cliente y las colas.
- Con el historial de ventas se puede **anticipar la demanda** y comprar mejor.
- Menos errores de registro implican menos reclamos y más confianza del cliente.

Pregunta 4. ¿Qué beneficios adicionales ofrece para ser más competitiva?

- **Control de vencimientos** y rotación de lotes (FEFO: primero en vencer, primero en salir).
- **Reportes e indicadores** para decidir qué comprar, qué promocionar y qué dejar de pedir.
- **Historial del cliente** para ofrecer atención personalizada y programas de fidelización.
- **Comparación de proveedores** y mejores precios de compra.
- **Acceso desde cualquier dispositivo** (PC, tablet, celular) para el dueño, aun fuera de la botica.
- **Escalabilidad**: agregar delivery, ventas en línea o una segunda sucursal.

Pregunta 5. ¿Qué métricas evalúan la efectividad del software?

Objetivo	Métrica (KPI)	Cómo medirla
Tiempo de respuesta	**Tiempo promedio de atención por cliente**	Cronometrar antes y después de implementar
Tiempo de respuesta	**Tiempo de carga de pantallas / API**	Lighthouse, logs del servidor (meta: menos de 2 s)
Errores de inventario	**% de diferencia entre stock físico y sistema**	Conteo cíclico vs. datos del sistema
Desabastecimiento	**Nº de quiebres de stock por mes**	Registro de productos agotados
Desabastecimiento	**% de alertas atendidas a tiempo**	Alertas generadas vs. reposiciones realizadas
Mermas	**Valor de productos vencidos por mes**	Reporte de bajas por vencimiento
Adopción	**Satisfacción del personal y del**	Encuesta corta (1 a 5)

cliente

---

3. Requisitos del sistema

3.1 Requisitos funcionales

Código	Requisito
RF01	El sistema permite iniciar sesión con usuario y contraseña.
RF02	El administrador gestiona usuarios y roles.
RF03	El sistema permite registrar, editar y desactivar productos.
RF04	El sistema registra ingresos de mercadería por lote y vencimiento.
RF05	El sistema registra ventas y descuenta el stock automáticamente.
RF06	El sistema genera alertas por stock bajo y por vencimiento próximo.
RF07	El sistema registra clientes y su historial de compras.
RF08	El sistema genera reportes de ventas e inventario.

3.2 Requisitos no funcionales (condiciones del caso)

Código	Requisito	Cómo se cumple
RNF01	Accesible desde dispositivos web	Aplicación React **responsive** (PC, tablet, móvil)
RNF02	Seguridad de los datos	HTTPS, JWT, contraseñas con bcrypt, roles, validación de entradas
RNF03	Interfaz intuitiva y rápida	POS con buscador, atajos de teclado, pocos clics
RNF04	Módulo de alertas	Servicio de alertas + notificaciones en tiempo real

---

4. Arquitectura propuesta

Arquitectura **cliente-servidor en 3 capas**: el frontend React consume una API REST; la API se conecta a la base de datos.



4.1 Tecnologías

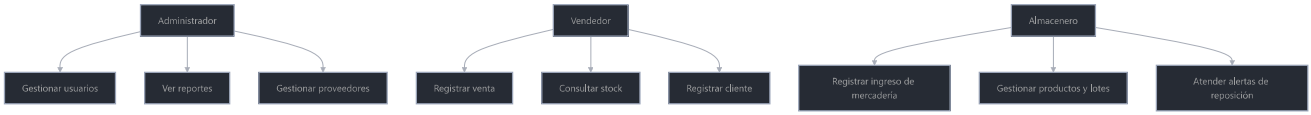
Capa	Tecnología	Motivo
Frontend	**React 18 + Vite**	Rápido, estándar de la industria

Rutas	**React Router**	Navegación entre módulos
Estado del servidor	**TanStack Query (React Query)**	Cache, recarga automática de datos
Estado global	**Context API** o **Zustand**	Sesión del usuario, carrito
Estilos	**Tailwind CSS** (o Material UI)	Diseño responsive rápido
Formularios	**React Hook Form**	Validación simple
HTTP	**Axios**	Interceptores para el token
Tiempo real	**Socket.IO client**	Alertas y stock en vivo
Backend	**Node.js + Express**	Mismo lenguaje (JavaScript) en todo el proyecto
BD	**PostgreSQL** (o MySQL)	Relacional, transacciones seguras
Seguridad	**JWT + bcrypt + helmet + cors**	Autenticación y protección
Despliegue	**Vercel/Netlify** (front), **Render/Railway** (back + BD)	Gratuitos para empezar

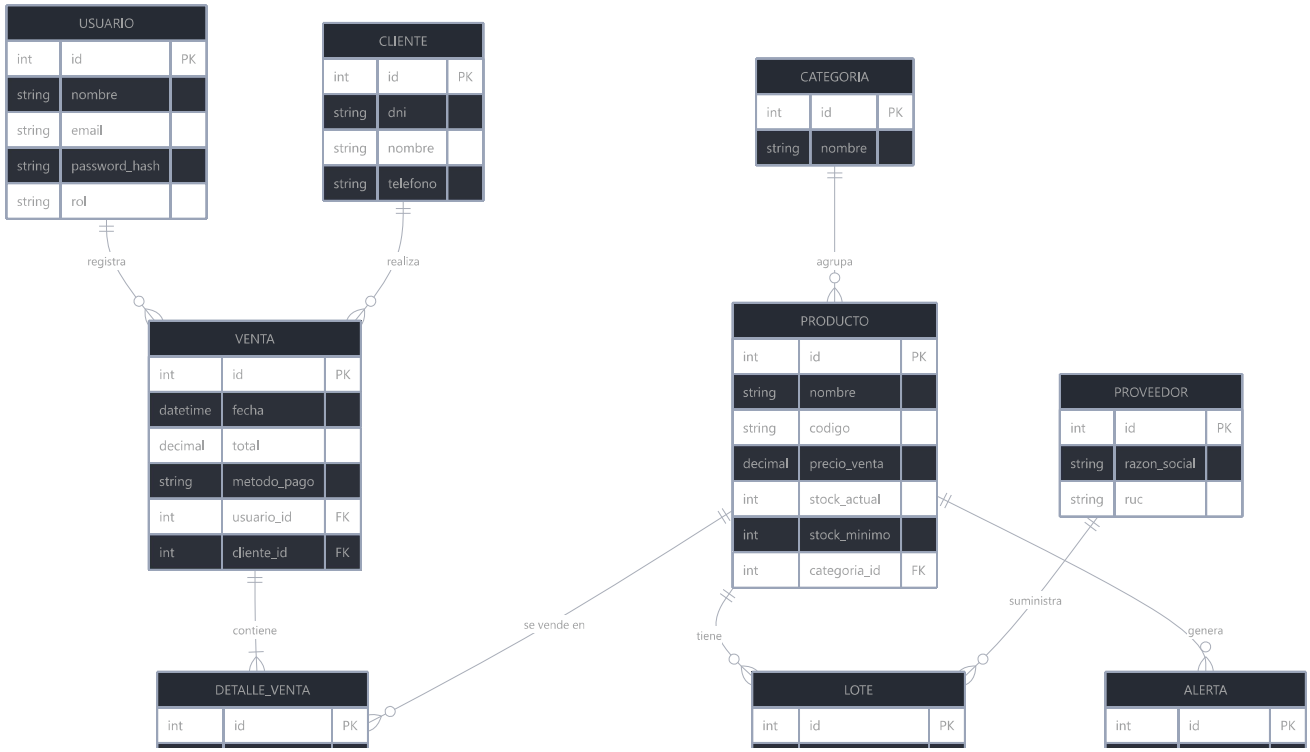
---

5. Diagramas

5.1 Casos de uso (simplificado por rol)



5.2 Modelo de datos (entidad-relación)



int	venta_id	FK
int	producto_id	FK
int	cantidad	
decimal	precio_unitario	

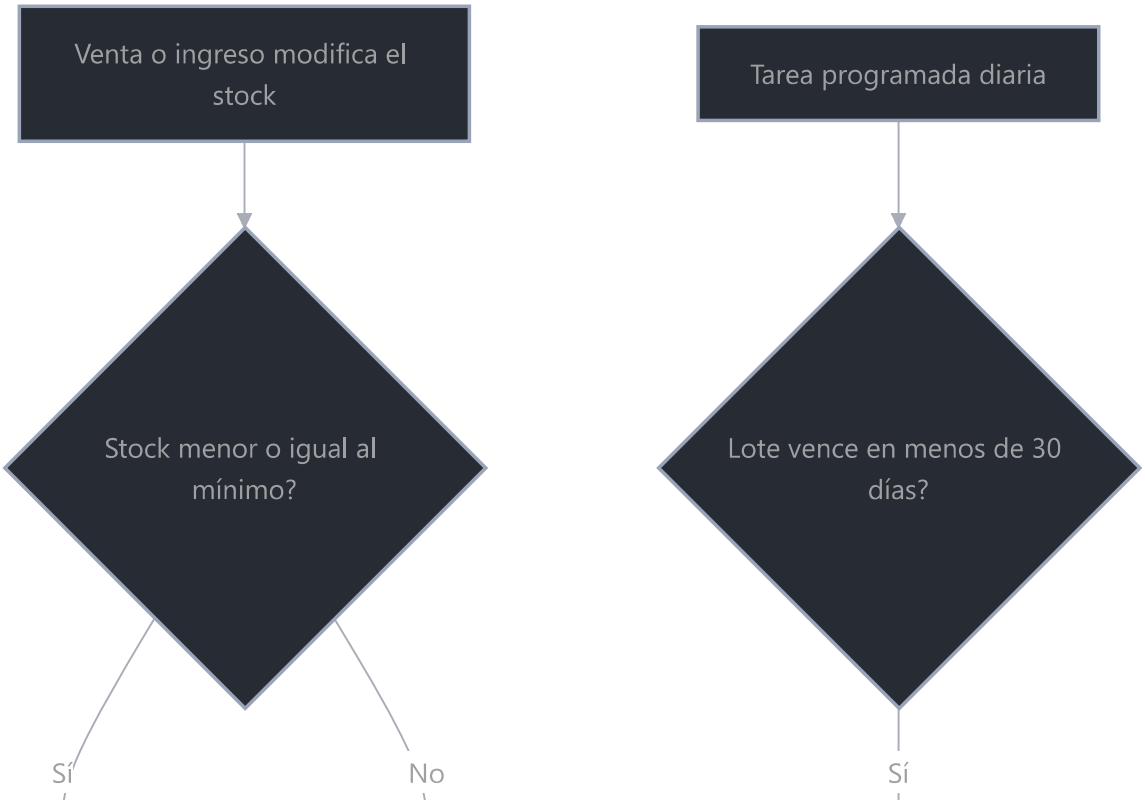
int	producto_id	FK
int	proveedor_id	FK
int	cantidad	
date	fecha_vencimiento	

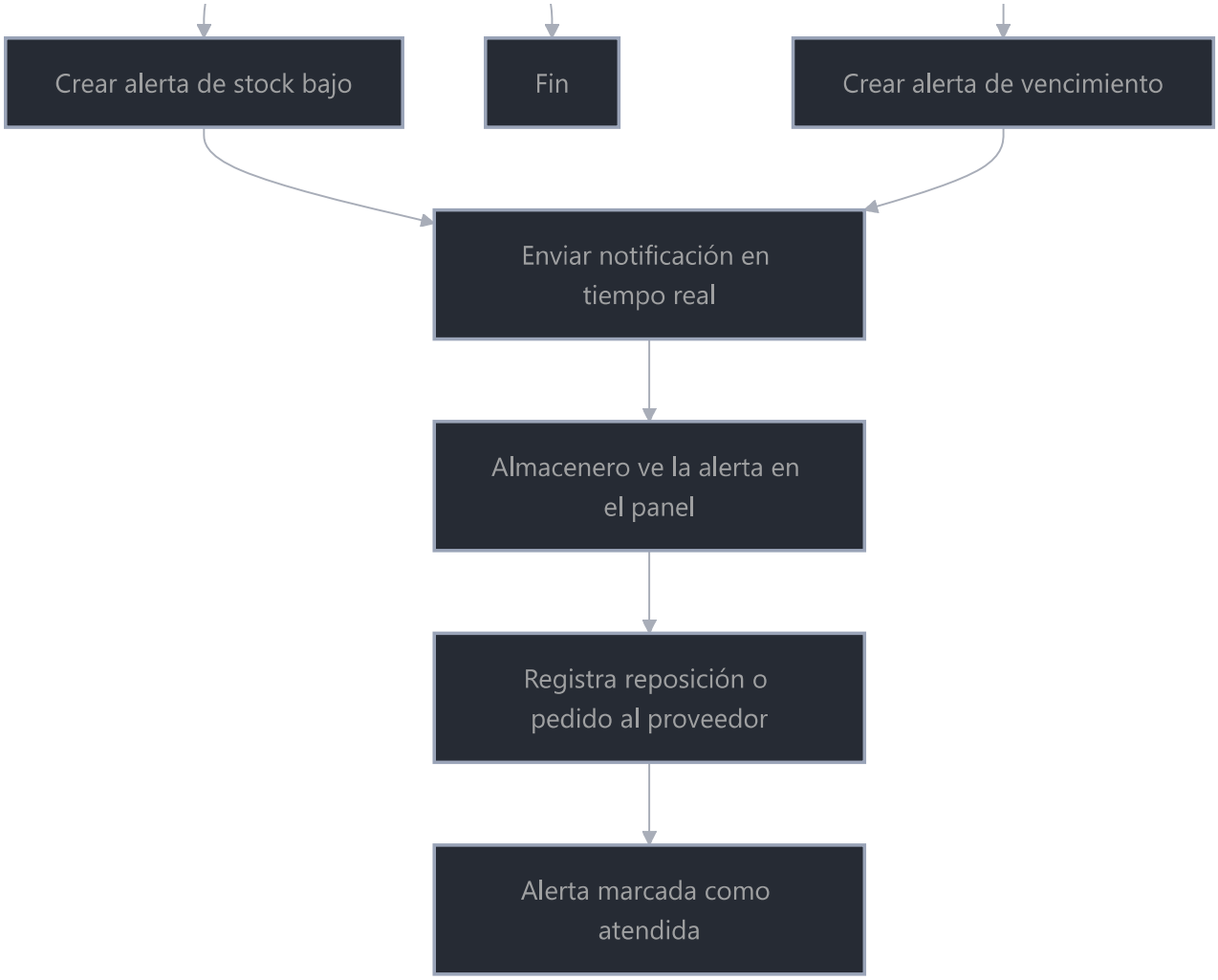
int	producto_id	FK
string	tipo	
string	estado	
datetime	fecha	

5.3 Secuencia: registrar una venta



5.4 Flujo del módulo de alertas





---

6. Paso a paso: construcción del Frontend con React

Paso 1. Preparar el entorno

Instalar **Node.js 18+** y **Git**. Verificar:

```
node -v
npm -v
git --version
```

Paso 2. Crear el proyecto React con Vite

```
npm create vite@latest botica-frontend -- --template react
cd botica-frontend
npm install
```

Paso 3. Instalar las librerías

```
npm install react-router-dom axios @tanstack/react-query react-hook-form socket.io-client
npm install -D tailwindcss @tailwindcss/vite
```

En `vite.config.js`:

```
import { defineConfig } from 'vite'
```

```
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

En `src/index.css`:

```
@import "tailwindcss";
```

Paso 4. Organizar la estructura de carpetas

```
src/
├── api/              # Configuración de Axios y funciones de acceso a la API
│   ├── axios.js
│   ├── productos.js
│   └── ventas.js
├── components/       # Componentes reutilizables
│   ├── Navbar.jsx
│   ├── Sidebar.jsx
│   ├── ProtectedRoute.jsx
│   └── AlertBell.jsx
├── context/
│   └── AuthContext.jsx
├── hooks/
│   └── useAlertas.js
├── pages/
│   ├── Login.jsx
│   ├── Dashboard.jsx
│   ├── Inventario.jsx
│   ├── Ventas.jsx        # POS
│   ├── Clientes.jsx
│   ├── Alertas.jsx
│   └── Reportes.jsx
├── App.jsx
└── main.jsx
```

Paso 5. Configurar Axios con el token JWT

`src/api/axios.js`

```
import axios from 'axios'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3000/api',
})

// Adjunta el token en cada petición
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// Si el token expira, cierra la sesión
```

```
// Si el token expira, cierra la sesión
api.interceptors.response.use(
  (res) => res,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token')
      window.location.href = '/login'
    }
    return Promise.reject(error)
  }
)

export default api
```

Crear el archivo `.env`:

```
VITE_API_URL=http://localhost:3000/api
```

Paso 6. Autenticación y rutas protegidas

`src/context/AuthContext.jsx`

```
import { createContext, useContext, useState } from 'react'
import api from '../api/axios'

const AuthContext = createContext(null)

export function AuthProvider({ children }) {
  const [usuario, setUsuario] = useState(() => {
    const guardado = localStorage.getItem('usuario')
    return guardado ? JSON.parse(guardado) : null
  })

  const login = async (email, password) => {
    const { data } = await api.post('/auth/login', { email, password })
    localStorage.setItem('token', data.token)
    localStorage.setItem('usuario', JSON.stringify(data.usuario))
    setUsuario(data.usuario)
  }

  const logout = () => {
    localStorage.clear()
    setUsuario(null)
  }

  return (
    <AuthContext.Provider value={{ usuario, login, logout }}>
      {children}
    </AuthContext.Provider>
  )
}

export const useAuth = () => useContext(AuthContext)
```

`src/components/ProtectedRoute.jsx`

```
import { Navigate } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'

export default function ProtectedRoute({ children, roles }) {
  const { usuario } = useAuth()

  if (!usuario) return <Navigate to="/login" replace />
  if (roles && !roles.includes(usuario.rol)) return <Navigate to="/" replace />

  return children
}
```

> **Importante para la sustentación:** ocultar una ruta en el frontend **no es seguridad real**. La verificación de roles debe repetirse siempre en el backend.

Paso 7. Configurar las rutas

`src/App.jsx`

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import { AuthProvider } from './context/AuthContext'
import ProtectedRoute from './components/ProtectedRoute'
import Layout from './components/Layout'
import Login from './pages/Login'
import Dashboard from './pages/Dashboard'
import Inventario from './pages/Inventario'
import Ventas from './pages/Ventas'
import Alertas from './pages/Alertas'
import Clientes from './pages/Clientes'
import Reportes from './pages/Reportes'

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<Login />} />
          <Route element={<ProtectedRoute><Layout /></ProtectedRoute>}>
            <Route path="/" element={<Dashboard />} />
            <Route path="/ventas" element={<Ventas />} />
            <Route path="/clientes" element={<Clientes />} />
            <Route path="/alertas" element={<Alertas />} />
            <Route
              path="/inventario"
              element={
                <ProtectedRoute roles={['admin', 'almacen']}>
                  <Inventario />
                </ProtectedRoute>
              }
            />
            <Route
              path="/reportes"
              element={
                <ProtectedRoute roles={['admin']}>
                  <Reportes />
                </ProtectedRoute>
              }
            />
```

```
          </Route>
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  )
}
```

`src/main.jsx` (agregar React Query):

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import App from './App'
import './index.css'

const queryClient = new QueryClient()

ReactDOM.createRoot(document.getElementById('root')).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
)
```

Paso 8. Módulo de Inventario

`src/api/productos.js`

```
import api from './axios'

export const getProductos = (buscar = '') =>
  api.get('/productos', { params: { buscar } }).then((r) => r.data)

export const crearProducto = (producto) =>
  api.post('/productos', producto).then((r) => r.data)

export const actualizarProducto = (id, producto) =>
  api.put(`/productos/${id}`, producto).then((r) => r.data)
```

`src/pages/Inventario.jsx` (tabla con indicador de stock bajo)

```
import { useState } from 'react'
import { useQuery } from '@tanstack/react-query'
import { getProductos } from '../api/productos'

export default function Inventario() {
  const [buscar, setBuscar] = useState('')
  const { data: productos = [], isLoading } = useQuery({
    queryKey: ['productos', buscar],
    queryFn: () => getProductos(buscar),
  })

  if (isLoading) return <p>Cargando inventario...</p>

  return (
    <div className="p-4">
      <h1 className="text-2xl font-bold mb-4">Inventario</h1>
```

```
        <input
          className="border rounded px-3 py-2 w-full max-w-md mb-4"
          placeholder="Buscar por nombre o código..."
          value={buscar}
          onChange={(e) => setBuscar(e.target.value)}
        />

        <div className="overflow-x-auto">
          <table className="w-full text-left border">
            <thead className="bg-gray-100">
              <tr>
                <th className="p-2">Código</th>
                <th className="p-2">Producto</th>
                <th className="p-2">Precio</th>
                <th className="p-2">Stock</th>
                <th className="p-2">Estado</th>
              </tr>
            </thead>
            <tbody>
              {productos.map((p) => {
                const bajo = p.stock_actual <= p.stock_minimo
                return (
                  <tr key={p.id} className="border-t">
                    <td className="p-2">{p.codigo}</td>
                    <td className="p-2">{p.nombre}</td>
                    <td className="p-2">S/ {Number(p.precio_venta).toFixed(2)}</td>
                    <td className="p-2">{p.stock_actual}</td>
                    <td className="p-2">
                      <span className={bajo ? 'text-red-600 font-semibold' : 'text-green-600'}>
                        {bajo ? 'Stock bajo' : 'Normal'}
                      </span>
                    </td>
                  </tr>
                )
              })}
            </tbody>
          </table>
        </div>
      </div>
    )
  }
```

Paso 9. Módulo de Ventas (POS): el más importante

El POS debe cumplir la condición "*interfaz intuitiva que reduzca el tiempo de atención*". Recomendaciones de diseño:

- Buscador **siempre enfocado** (autofocus) para usar lector de código de barras.
- Agregar al carrito con **Enter** o un clic.
- Botones grandes, totales visibles todo el tiempo.
- Pantalla dividida: **búsqueda a la izquierda, carrito a la derecha.**

`src/pages/Ventas.jsx`

```
import { useState, useMemo } from 'react'
```

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { getProductos } from '../api/productos'
import api from '../api/axios'

export default function Ventas() {
  const [buscar, setBuscar] = useState('')
  const [carrito, setCarrito] = useState([])
  const [metodoPago, setMetodoPago] = useState('efectivo')
  const queryClient = useQueryClient()

  const { data: resultados = [] } = useQuery({
    queryKey: ['productos', buscar],
    queryFn: () => getProductos(buscar),
    enabled: buscar.length >= 2,
  })

  const agregar = (producto) => {
    setCarrito((actual) => {
      const existe = actual.find((i) => i.id === producto.id)
      if (existe) {
        if (existe.cantidad >= producto.stock_actual) return actual // no exceder stock
        return actual.map((i) =>
          i.id === producto.id ? { ...i, cantidad: i.cantidad + 1 } : i
        )
      }
      return [...actual, { ...producto, cantidad: 1 }]
    })
    setBuscar('')
  }

  const quitar = (id) => setCarrito((c) => c.filter((i) => i.id !== id))

  const total = useMemo(
    () => carrito.reduce((suma, i) => suma + i.precio_venta * i.cantidad, 0),
    [carrito]
  )

  const cobrar = useMutation({
    mutationFn: () =>
      api.post('/ventas', {
        metodo_pago: metodoPago,
        items: carrito.map((i) => ({ producto_id: i.id, cantidad: i.cantidad })),
      }),
    onSuccess: () => {
      setCarrito([])
      queryClient.invalidateQueries({ queryKey: ['productos'] }) // refresca stock
      alert('Venta registrada correctamente')
    },
    onError: (e) => alert(e.response?.data?.mensaje || 'Error al registrar la venta'),
  })

  return (
    <div className="grid md:grid-cols-2 gap-4 p-4">
      {/* Búsqueda */}
      <section>
        <input
          autoFocus
          className="border rounded px-3 py-3 w-full text-lg"
```

```
            placeholder="Buscar producto o escanear código..."
            value={buscar}
            onChange={(e) => setBuscar(e.target.value)}
          />
          <ul className="mt-2 divide-y border rounded">
            {resultados.map((p) => (
              <li key={p.id}>
                <button
                  onClick={() => agregar(p)}
                  disabled={p.stock_actual === 0}
                  className="w-full flex justify-between p-3 hover:bg-blue-50 disabled:opacity-40"
                >
                  <span>{p.nombre}</span>
                  <span>S/ {Number(p.precio_venta).toFixed(2)} · Stock: {p.stock_actual}</span>
                </button>
              </li>
            ))}
          </ul>
        </section>

        {/* Carrito */}
        <section className="border rounded p-4">
          <h2 className="text-xl font-bold mb-2">Carrito</h2>
          {carrito.length === 0 && <p className="text-gray-500">Sin productos</p>}
          {carrito.map((i) => (
            <div key={i.id} className="flex justify-between items-center py-1">
              <span>{i.cantidad} × {i.nombre}</span>
              <span>
                S/ {(i.precio_venta * i.cantidad).toFixed(2)}
                <button onClick={() => quitar(i.id)} className="ml-3 text-red-600">✕</button>
              </span>
            </div>
          ))}

          <div className="border-t mt-4 pt-4">
            <select
              className="border rounded p-2 mb-3 w-full"
              value={metodoPago}
              onChange={(e) => setMetodoPago(e.target.value)}
            >
              <option value="efectivo">Efectivo</option>
              <option value="tarjeta">Tarjeta</option>
              <option value="yape">Yape / Plin</option>
            </select>
            <p className="text-2xl font-bold mb-3">Total: S/ {total.toFixed(2)}</p>
            <button
              onClick={() => cobrar.mutate()}
              disabled={carrito.length === 0 || cobrar.isPending}
              className="w-full bg-blue-600 text-white py-3 rounded text-lg disabled:opacity-50"
            >
              {cobrar.isPending ? 'Procesando...' : 'Cobrar'}
            </button>
          </div>
        </section>
      </div>
    )
  }
```

Paso 10. Módulo de Alertas en tiempo real

`src/hooks/useAlertas.js`

```
import { useEffect, useState } from 'react'
import { io } from 'socket.io-client'
import { useQuery, useQueryClient } from '@tanstack/react-query'
import api from '../api/axios'

const SOCKET_URL = import.meta.env.VITE_SOCKET_URL || 'http://localhost:3000'

export function useAlertas() {
  const queryClient = useQueryClient()
  const [nuevaAlerta, setNuevaAlerta] = useState(null)

  const { data: alertas = [] } = useQuery({
    queryKey: ['alertas'],
    queryFn: () => api.get('/alertas?estado=pendiente').then((r) => r.data),
  })

  useEffect(() => {
    const socket = io(SOCKET_URL, { auth: { token: localStorage.getItem('token') } })

    socket.on('alerta:nueva', (alerta) => {
      setNuevaAlerta(alerta)
      queryClient.invalidateQueries({ queryKey: ['alertas'] })
    })

    return () => socket.disconnect()
  }, [queryClient])

  return { alertas, nuevaAlerta }
}
```

`src/components/AlertBell.jsx` (campana con contador en la barra superior)

```
import { Link } from 'react-router-dom'
import { useAlertas } from '../hooks/useAlertas'

export default function AlertBell() {
  const { alertas } = useAlertas()

  return (
    <Link to="/alertas" className="relative text-2xl">
      🔔
      {alertas.length > 0 && (
        <span className="absolute -top-2 -right-2 bg-red-600 text-white text-xs rounded-full px-1.
          {alertas.length}
        </span>
      )}
    </Link>
  )
}
```

Paso 11. Dashboard y Reportes

- **Dashboard:** tarjetas con *ventas del día*, *productos con stock bajo*, *productos por vencer* y un gráfico de ventas de la semana.
- Para los gráficos instalar `npm install recharts`.
- **Reportes:** filtro por rango de fechas, tabla de ventas, productos más vendidos y botón de exportar a CSV/PDF.

Paso 12. Diseño responsive y usabilidad

- Usar clases responsive de Tailwind (`md:`, `lg:`) para que funcione en celular y tablet.
- Mensajes claros de error y confirmación.
- Colores semánticos: **rojo** (stock bajo/agotado), **amarillo** (por vencer), **verde** (normal).
- Estados de carga (*loading*) y vacío en todas las pantallas.
- Atajos de teclado en el POS (Enter para agregar, F2 para cobrar).

---

7. Backend: resumen de lo necesario

Aunque el foco del trabajo es React, el frontend necesita una API. Estructura mínima:

```
botica-backend/
├── src/
│   ├── config/db.js
│   ├── middlewares/auth.js        # Verifica JWT y rol
│   ├── routes/                    # auth, productos, ventas, clientes, alertas
│   ├── controllers/
│   ├── services/alertas.js        # Lógica de alertas
│   └── app.js
└── .env
```

7.1 Endpoints principales

Método	Ruta	Descripción	Rol
POST	`/api/auth/login`	Iniciar sesión	Público
GET	`/api/productos?buscar=`	Listar/buscar productos	Todos
POST	`/api/productos`	Crear producto	Admin, Almacén
PUT	`/api/productos/:id`	Editar producto	Admin, Almacén
POST	`/api/lotes`	Registrar ingreso de mercadería	Almacén
POST	`/api/ventas`	Registrar venta	Vendedor, Admin
GET	`/api/ventas?desde=&hasta=`	Consultar ventas	Admin
GET	`/api/clientes?dni=`	Buscar cliente	Todos
GET	`/api/alertas?estado=`	Listar alertas	Todos
PATCH	`/api/alertas/:id`	Marcar alerta atendida	Almacén, Admin

PATCH	/api/alertas/:id	Marcar alerta atendida	Almacén, Admin
GET	/api/reportes/ventas	Reportes	Admin

7.2 Lógica clave: registrar venta con transacción

Es fundamental usar una **transacción** para que la venta y el descuento de stock se guarden juntos o no se guarden:

```
// controllers/ventas.js (resumen)
export async function registrarVenta(req, res) {
  const client = await pool.connect()
  try {
    await client.query('BEGIN')

    const venta = await client.query(
      'INSERT INTO venta (usuario_id, metodo_pago, total) VALUES ($1,$2,0) RETURNING id',
      [req.usuario.id, req.body.metodo_pago]
    )

    let total = 0
    for (const item of req.body.items) {
      const { rows } = await client.query(
        'SELECT precio_venta, stock_actual, stock_minimo FROM producto WHERE id=$1 FOR UPDATE',
        [item.producto_id]
      )
      const prod = rows[0]
      if (!prod || prod.stock_actual < item.cantidad) {
        throw new Error('Stock insuficiente')
      }

      await client.query(
        'INSERT INTO detalle_venta (venta_id, producto_id, cantidad, precio_unitario) VALUES ($1,$
        [venta.rows[0].id, item.producto_id, item.cantidad, prod.precio_venta]
      )
      await client.query(
        'UPDATE producto SET stock_actual = stock_actual - $1 WHERE id=$2',
        [item.cantidad, item.producto_id]
      )

      total += prod.precio_venta * item.cantidad

      if (prod.stock_actual - item.cantidad <= prod.stock_minimo) {
        // crear alerta y emitir por socket.io
      }
    }

    await client.query('UPDATE venta SET total=$1 WHERE id=$2', [total, venta.rows[0].id])
    await client.query('COMMIT')
    res.status(201).json({ id: venta.rows[0].id, total })
  } catch (e) {
    await client.query('ROLLBACK')
    res.status(400).json({ mensaje: e.message })
  } finally {
    client.release()
  }
}
```

8. Seguridad de los datos (condición 1 del caso)

Medida	Dónde	Detalle
HTTPS	Despliegue	Obligatorio en producción
Contraseñas cifradas	Backend	`bcrypt` (nunca texto plano)
JWT con expiración	Backend/Frontend	Tokens de corta duración
Control de roles	Backend (principal) y Frontend	Admin / Vendedor / Almacén
Validación de entradas	Backend	`zod` o `express-validator`
Consultas parametrizadas	Backend	Evita SQL Injection (`$1, $2`)
Cabeceras seguras	Backend	`helmet`
CORS restringido	Backend	Solo el dominio del frontend
Límite de intentos de login	Backend	`express-rate-limit`
Variables de entorno	Ambos	Nunca subir `.env` a GitHub
Copias de seguridad	Base de datos	Backups automáticos diarios

9. Pruebas y despliegue

9.1 Pruebas

- **Manuales:** flujo completo de venta, ingreso de lote, generación de alerta.
- **Frontend:** `Vitest + React Testing Library` para componentes clave (carrito, login).
- **API:** `Postman` o `Thunder Client` para probar cada endpoint.
- **Usabilidad:** pedir a 2 o 3 personas que realicen una venta y medir el tiempo.

9.2 Despliegue

```
# Generar la versión de producción del frontend
npm run build
```

1. Subir el código a **GitHub** (repos separados o monorepo).
2. **Frontend** → Vercel o Netlify (configurar `VITE_API_URL` con la URL real del backend).
3. **Backend + BD** → Render o Railway (configurar variables de entorno).
4. Verificar HTTPS y probar desde un celular.

10. Plan de trabajo para el equipo

Semana	Actividad	Entregable
1	Análisis, requisitos, diagramas, prototipos (Figma o papel)	Documento de requisitos y diagramas
2	Base de datos y backend base (auth, productos)	API funcionando en local
3	Frontend: login, layout, rutas, inventario	Pantallas de acceso e inventario
4	Backend y frontend de ventas (POS)	Ventas con descuento de stock
5	Alertas en tiempo real, dashboard y reportes	Módulo de alertas operativo
6	Pruebas, correcciones, despliegue, presentación	Sistema en línea e informe final

Reparto sugerido para 4 integrantes:

1. **Frontend A:** login, layout, inventario, clientes.
2. **Frontend B:** POS de ventas, alertas, dashboard.
3. **Backend:** API, base de datos, seguridad, alertas.
4. **QA y documentación:** pruebas, diagramas, informe, despliegue, presentación.

---

11. Checklist del entregable

- ⬛ Introducción y objetivos del proyecto
- ⬛ Respuestas a las 5 preguntas guía
- ⬛ Requisitos funcionales y no funcionales
- ⬛ Diagramas: arquitectura, casos de uso, entidad-relación, secuencia, flujo de alertas
- ⬛ Prototipos o capturas de las pantallas (Login, Dashboard, Inventario, POS, Alertas)
- ⬛ Explicación del stack tecnológico (React y justificación)
- ⬛ Medidas de seguridad aplicadas
- ⬛ Métricas de evaluación (KPIs)
- ⬛ Plan de despliegue
- ⬛ Conclusiones y recomendaciones
- ⬛ Enlace al repositorio y al sistema desplegado

---

12. Conclusiones sugeridas

1. Un sistema web centralizado elimina la dependencia del control manual y reduce los errores de inventario.
2. Las alertas automáticas permiten reponer a tiempo y evitar el desabastecimiento.
3. React permite construir una interfaz rápida, modular y reutilizable, ideal para un punto de venta que debe atender con rapidez.
4. La seguridad (JWT, roles, HTTPS, cifrado) protege la información sensible del negocio.
5. Las métricas propuestas permiten comprobar objetivamente la mejora tras la implementación.

Documento elaborado como guía de resolución del Trabajo Final del curso Fullstack Developer Software – SENATI.